

18. Se souvenir de l'état d'avant

Compter un appui, et non soixante par seconde.

1. Ce qu'il faut comprendre

Sans les deux lignes du milieu, `combien` monterait de **soixante par seconde** tant qu'on garde le doigt dessus : la boucle tourne soixante fois, et le bouton est enfoncé à chacun de ces tours.

Garder l'état d'avant, et ne compter que le **changement** — c'est ce qu'on appelle un front. On s'en sert pour tout ce qui doit arriver **une fois** : sauter, tirer, valider un menu.

La leçon « Le front, ou pourquoi un appui compte trois fois » montre ce qui arrive quand on l'oublie.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **598 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         69 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   76 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
LireManette    37 o ← bouton() - lit les huit boutons
Tuiles         240 o ← CopierTuiles - 15 lettres sur 43
"APPUIS"       6 o ← texte() - un texte gravé
```

Lettres gravées : AIPSU0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```
bouton() × 1    12 o sur place, dans main() + LireManette (37 o)
texte() × 1     11 o sur place, dans main() + EcrireTexte (10 o)
nombre() × 1    10 o sur place, dans main() + EcrireNombre (33 o)
image() × 1      3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : compte les appuis sur A. Un appui = +1, même si on garde
// le doigt dessus longtemps.
//
// Le problème : bouton(A) reste à 1 TANT QUE A est enfoncée (plusieurs images).
// La solution : se souvenir de l'état d'avant, et ne compter qu'au moment
// où A passe de « relâchée » à « enfoncée ». On appelle cela le FRONT.

#include <bouton> // lit un bouton de la manette
#include <texte>   // écrit un texte à l'écran
#include <nombre> // écrit un nombre en chiffres
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() { // Le jeu commence ici.
    uint8_t combien = 0; // Le nombre d'appuis.
    uint8_t avant = 0;   // A était-elle enfoncée au tour d'avant ?

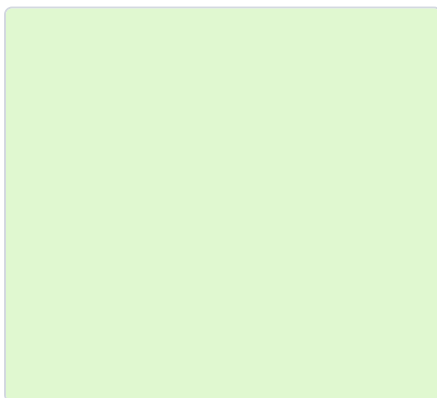
    while (true) { // Répète pour toujours :
        uint8_t maintenant = bouton(A); // A enfoncée maintenant ? (1 ou 0)

        // Le FRONT : enfoncé maintenant, et pas au tour d'avant.
        // !avant = « NON avant » : vrai si avant vaut 0.
        // && = ET : les deux doivent être vrais.
        if (maintenant && !avant) combien++;
        avant = maintenant; // On retient pour le tour suivant.

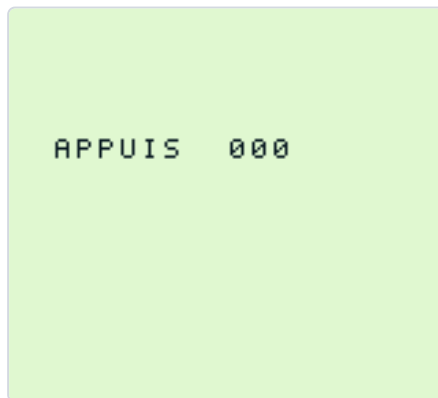
        texte(2, 6, "APPUIS");
        nombre(10, 6, combien); // Affiche le nombre d'appuis.
        image(); // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



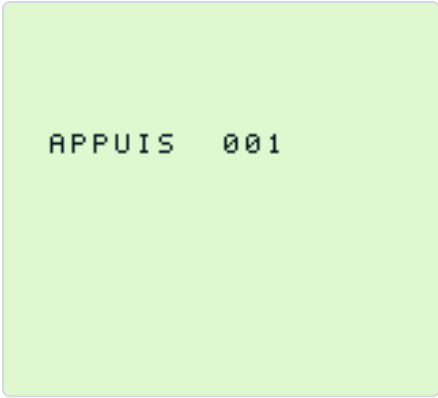
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



APPUIS 001

A — l'écran a changé

5. Ce qu'on doit voir

Le compteur monte de un par appui, même si l'on tient la touche.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ un appui long ne compte qu'une fois (1)
- ✓ deux appuis de plus en font trois (3)
- ✓ et le compte est affiché

Ensuite : **19. Un choix, puis plusieurs** — chapitre 3 : Retenir, et réagir. · [gameboy3](#) — écrire une cartouche Game Boy en C++